

# TavernBlink

## 从立项到落地实施报告

### 基于 NETransparentProxyProvider 的 macOS 原生拔线工具

**技术路线：**NETransparentProxyProvider + Network System Extension

**目标发行：**Developer ID 签名、公证后的 DMG（非 Mac App Store）

**建议基线：**macOS 13+；Universal 2；单机、无后台服务

**报告版本：**v1.0 · 2026-07-28

**面向对象：**TavernBlink 单人开发与小规模测试

**最终判断** 该路线在 Apple 公共 API 边界内具备产品级可行性：TavernBlink 可以把透明代理系统扩展内嵌进一个 .app，在不依赖 Clash/Mihomo、无需 root、无需解密 TLS 的情况下，持有并主动关闭炉石 TCP flow。真正决定项目能否继续的不是 UI，而是四个前置门槛：entitlement/签名、flow 身份识别、稳定双向 relay、关闭后是否能快速触发游戏重连。

- **MVP** 只做手动“立即拔线”；不做战斗自动识别，不读网络明文，不注入游戏进程。
- 先关闭所有由炉石签名身份发起的活动 TCP flow，后续再用实测数据缩小到真正的对局 flow。
- 采用系统提供的 provider messaging 传递命令；扩展和主程序共享的持久数据仅限配置与诊断状态。
- 公开分发前必须完成 Developer ID 签名与 Apple 公证；公证不代表 Apple 或 Blizzard 对用途的认可。

**重要：**利用断线跳过动画可能违反 Blizzard 规则并带来账号处罚风险；第 11 节给出发布边界。

## 执行摘要

现有 hearthstone\_skipper 本质上是 Clash/Mihomo 的控制端：它读取 Clash 的连接列表，按 Hearthstone 进程路径筛选连接，再向 /connections/{id} 发送 DELETE。真正拥有 socket 并终止连接的是代理核心，而不是 skipper 自身。TavernBlink 要做的，就是用 Apple 的 Transparent Proxy system extension 原生替代这层“连接所有权”。<sup>(1)</sup>

Apple 明确允许 NETransparentProxyProvider 对匹配流量选择“由扩展处理”或“交回系统处理”。因此 TavernBlink 可以让 Network Extension 看到广义的出站 TCP flow，在 handleNewFlow 中用 sourceAppSigningIdentifier 精确选择炉石；非目标 flow 返回 false，目标 flow 才进入代理 relay。<sup>(2)</sup>

目标 flow 被接管后，系统扩展用 NWConnection 连接原始远端端点，并在 NEAppProxyTCPFlow 与 NWConnection 之间复制原始字节。点击按钮时，主程序通过 sendProviderMessage 发送 disconnectNow，扩展关闭 flow 的读写方向并取消上游连接。Apple API 保证这些“关闭”操作存在；但它是否在当前炉石版本上以足够快的速度触发重连，必须由 Phase 0 实机 PoC 验证。<sup>(3)</sup>

## 这份报告给出的决定

- 选择 NETransparentProxyProvider，不再并行维护 PF、Content Filter 或内置 Mihomo 方案。
- 选择“单一菜单栏 App + 内嵌 Network System Extension”的产品形态。
- 选择直接分发：Developer ID + notarization + DMG；暂不以 Mac App Store 为首发渠道。
- 选择 TCP-only MVP；只有实测证明对局依赖 UDP 时才扩展 NEAppProxyUDPFlow。
- 选择按签名身份识别目标 App；不把安装路径、目标 IP 或单一端口作为唯一判断。
- 选择“先平台门槛、后完整 UI”的开发顺序，避免在 entitlement 不可用时浪费数周。

## 目录与阅读顺序

1. 项目定义与验收目标 — 明确做什么、怎样算成功
2. 现有方案与 TavernBlink 的差异 — 解释为什么不再依赖 Clash
3. MVP 边界与版本路线 — 控制首版范围
4. 总体架构 — 主程序、系统扩展和对局连接如何协作

- 5. 核心系统设计 — flow 匹配、relay、命令与断开语义
- 6. Xcode 工程与权限配置 — targets、entitlements、Info.plist、bundle IDs
- 7. 从零实施路线图 — 逐阶段产物和 Go/No-Go 门槛
- 8. 测试与验收 — 真实 Mac、真实游戏与网络安全回归
- 9. 首次安装与产品体验 — 系统审批、状态机、用户提示
- 10. CI/CD 与发布 — 签名、公证、DMG、GitHub Release
- 11. 风险与合规 — 技术风险、EULA 和发布决策
- 12. 工期、资源与 Definition of Done — 可执行时间表与最终清单

**证据边界** 报告把“Apple API 已确认”和“必须由炉石实测确认”分开。API 可用性、flow-copying、关闭方法、provider messaging、System Extension 和 notarization 均有官方依据；对局使用哪条 flow、关闭后重连速度、是否需要关闭多条连接属于工程假设，不能用文档代替实测。

# 1. 项目定义与验收目标

## 1.1 产品定义

TavernBlink 是一个原生 macOS 菜单栏工具。用户完成一次系统扩展与网络代理授权后，在炉石传说酒馆战棋进入战斗时点击“立即拔线”，工具主动关闭当前炉石网络会话，使客户端进入自动重连流程。产品目标是跳过仍在本地播放的战斗动画，并尽快重新同步服务器状态。

产品不读取或修改游戏内存，不向炉石进程注入代码，不解析或解密 TLS，不安装第三方代理核心，也不更改全机网卡、路由或 DNS。它仅代理由目标签名身份发起的 flow，并在用户明确点击时关闭这些 flow。

## 1.2 可量化成功标准

表 1 MVP 成功指标与验收口径

| 维度 | 目标                                   | 验收方法           |
|----|--------------------------------------|----------------|
| 功能 | 单击后关闭至少一条活动炉石 TCP flow，并观察到新 flow 建立 | 扩展结构化日志 + 游戏画面 |

| 维度  | 目标                                                                      | 验收方法                           |
|-----|-------------------------------------------------------------------------|--------------------------------|
| 延迟  | 按钮到扩展收到命令 $p95 \leq 200\text{ ms}$ ；到旧 flow 关闭 $p95 \leq 500\text{ ms}$ | 本机 monotonic clock 打点，连续 50 次  |
| 重连  | 有效对局中，大多数测试在 5 秒内进入重连或恢复                                                | 至少 30 场实测；记录成功率与时间             |
| 隔离  | 浏览器、下载、视频会议等非炉石流量不被代理 relay 或主动关闭                                       | 并行回归测试 + flow 日志               |
| 性能  | 待机 CPU 平均 $< 1\%$ ；对局 relay CPU 平均 $< 5\%$ ；扩展内存 $< 100\text{ MB}$      | Activity Monitor / Instruments |
| 稳定性 | 连续运行 2 小时、重复 100 次命令，无死锁、无无限缓存、无系统网络中断                                  | 压力与故障注入测试                      |
| 交付  | DMG 已签名、公证并 stapled；新用户可按引导完成安装和卸载                                      | 干净用户账户 / 第二台 Mac               |

1.3 必须在 Phase 0 回答的四个问题

- 1. 开发团队的 App ID、profiles 和签名证书能否实际携带 app-proxy-provider-systemextension entitlement，并成功激活系统扩展？
- 2. NETransparentProxyManager 能否保存、启用并启动该 provider，且主程序能观察正确状态？
- 3. 运行炉石时，flow.metadata.sourceAppSigningIdentifier 是否稳定可用，并能与其他 Battle.net 进程区分？
- 4. 对已代理的真实对局 TCP flow 同时执行 closeReadWithError、closeWriteWithError 和 NWConnection.cancel() 后，炉石是否及时重连？

**Go / No-Go** 四项全部通过才进入完整产品开发。若第 1 项失败，先解决 Apple 账户与 entitlement；若第 3 项失败，增加 audit token 与签名校验回退；若第 4 项失败，则这条路线对 TavernBlink 的核心用途不成立，应停止，而不是继续堆 UI。

## 2. 现有方案与 TavernBlink 的差异

### 2.1 hearthstone\_skipper 的真实工作边界

hearthstone\_skipper 的 README 要求 Clash/Mihomo 接管炉石流量，暴露 external controller，并开启进程识别。源码中的 skip() 先 GET 连接列表，再检查 metadata.processPath 是否以 Hearthstone 可执行文件路径结尾，并额外要求 metadata.host == ""；随后对匹配连接发送 DELETE。换言之，它是控制器，不是连接代理。<sup>(2)</sup>

```
GET /connections
match metadata.processPath and metadata.host
DELETE /connections/{connection-id}
```

这个设计轻量，但把三个关键能力外包给第三方核心：截获连接、识别发起进程、主动终止连接。TavernBlink 的原生路线必须在自己的 Network System Extension 内重新实现这三项能力。

### 2.2 TavernBlink 的替代关系

表 2 从 Clash 方案到 Apple 原生方案的责任映射

| 原能力  | hearthstone_skipper          | TavernBlink                                   |
|------|------------------------------|-----------------------------------------------|
| 流量入口 | Clash/Mihomo TUN             | NETransparentProxyProvider + included rules   |
| 进程识别 | Mihomo processPath           | sourceAppSigningIdentifier，必要时 audit token 回退 |
| 连接持有 | 代理核心内部 socket                | NEAppProxyTCPFlow + NWConnection              |
| 断开命令 | HTTP DELETE                  | sendProviderMessage(.disconnectNow)           |
| 断开动作 | Mihomo 删除连接                  | 关闭 flow 读写并取消上游连接                             |
| 配置依赖 | external_controller / secret | 系统扩展授权与 NE manager 配置                         |

### 2.3 为什么不照搬旧筛选条件

- 安装路径可能改变，Rosetta 或未来原生二进制也可能改变可执行文件布局；路径只能用于诊断。
- host == "" 是代理实现细节，不是对局协议契约；它已经导致兼容性脆弱。
- 目标 IP、CDN 和端口可能随地区或补丁改变；首版不应硬编码单一端点。

- 签名 identifier 是更稳定的一级条件；flow 年龄、字节数、端口和活跃度只能作为二级排序。

### 3. MVP 边界与版本路线

#### 3.1 v0.1.0 必须包含

- 菜单栏状态：未安装、等待批准、未启动、就绪、无目标连接、断开成功、错误。
- 系统扩展安装/更新引导，以及 Transparent Proxy 配置保存、启用和启动。
- 炉石 App 身份发现与验证；保存签名 identifier，而不是仅保存路径。
- TCP 双向透明 relay；不解密 TLS，不修改 payload。
- 手动“立即拔线”命令；防抖、幂等、结果回执。
- 本地结构化诊断日志、导出诊断包、彻底禁用与卸载说明。
- Developer ID 签名、公证和可验证的 DMG。

#### 3.2 明确排除

- 自动判断战斗开始、读 Power.log、窗口识别或快捷键自动化。
- UDP relay，除非 Phase 0 证明当前对局实际需要。
- 账号系统、云端后台、遥测服务器、自动更新框架。
- 游戏内存访问、二进制注入、反作弊绕过或隐藏工具存在。
- Mac App Store 首发、付费系统或大规模推广。

#### 3.3 后续版本

表 3 建议版本切分

| 版本           | 目标                    | 进入条件             |
|--------------|-----------------------|------------------|
| v0.1.0 Alpha | 开发机完成手动断开与重连          | Phase 0-3 全部通过   |
| v0.2.0 Beta  | 5-20 名测试者；完善授权、诊断和兼容性 | 100 次压力测试与网络回归通过 |
| v0.3.0       | 连接评分，仅关闭最可能的对局 flow   | 积累足够匿名本地样本       |

| 版本     | 目标                        | 进入条件       |
|--------|---------------------------|------------|
| v1.0.0 | 稳定签名、公证、升级和卸载体验           | 发布门槛全部满足   |
| 未来可选   | 日志触发自动拔线、全局快捷键、Sparkle 更新 | 单独合规评审后再决定 |

3.4 范围冻结规则

- Phase 0 没有证明 UDP 必要性之前，MVP 保持 TCP-only。
- 自动触发、日志识别、全局快捷键和更新框架都不得进入首个可用版本。
- 每项新增功能都必须保护四个前置门槛：授权可得、非目标流量无影响、断开后快速重连、可签名公证。
- 远端 IP、域名或端口只能作为诊断和排序信号，不能成为唯一硬编码匹配条件。
- 首版保持单机、无账号、无后台服务，避免在核心网络链路尚未稳定时扩大隐私与运维面。

**变更控制** 任何超出 v0.1.0 范围的需求先进入 backlog；只有在 Phase 0-3 的退出门槛不受影响时，才允许排入后续迭代。

4. 总体架构

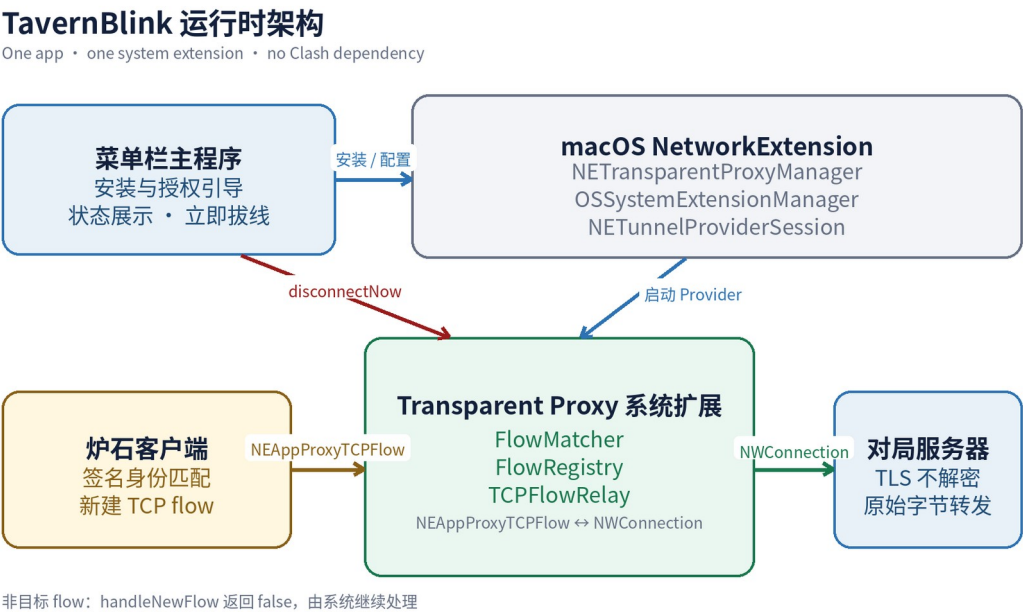


图 1 TavernBlink 的主程序、NetworkExtension、系统扩展与对局连接关系

4.1 组件职责

表 4 运行时组件与边界

| 组件                          | 核心职责                                     | 不负责          |
|-----------------------------|------------------------------------------|--------------|
| Menu Bar App                | 安装引导、状态、用户命令、诊断导出                        | 直接访问网络 flow  |
| System Extension Controller | 提交激活/替换/停用请求, 处理审批状态                     | 保存 NE 配置     |
| Proxy Manager Controller    | 加载、保存、启用和启动<br>NETransparentProxyManager | 转发 TCP 数据    |
| Provider Messenger          | 把结构化命令发送给运行中的 provider                   | 持久化大体积数据     |
| Transparent Proxy Provider  | 应用网络设置、接收 flow、处理命令、生命周期                 | UI 与自动更新     |
| Flow Matcher                | 按签名身份判断目标 flow                           | 解析 TLS 或游戏协议 |
| TCP Flow Relay              | 建立 NWConnection、双向复制、背压与关闭               | 决定何时拔线       |
| Flow Registry               | 保存活动 relay、汇总状态、原子断开                     | 长期保存用户流量     |



## 4.2 控制面与数据面

控制面由主程序、NE manager 和 provider messaging 组成，数据量小、要求幂等。数据面完全位于系统扩展：每个目标 NEAppProxyTCPFlow 对应一个 TCPFlowRelay 和一个上游 NWConnection。两者不能混在同一状态机里，否则 UI 重试或状态刷新很容易误关连接。

- 控制命令：status、disconnectNow、updateTargetIdentity、exportDiagnostics。
- provider 回执：协议版本、当前状态、目标 flow 数量、关闭数量、错误码、单调时间戳。
- 数据面内容：内存中短暂存在的原始字节块；不落盘、不分析、不上传。
- 共享持久数据：签名 identifier、用户偏好、最近错误摘要；不保存远端 payload。

## 4.3 一次断开的事件顺序

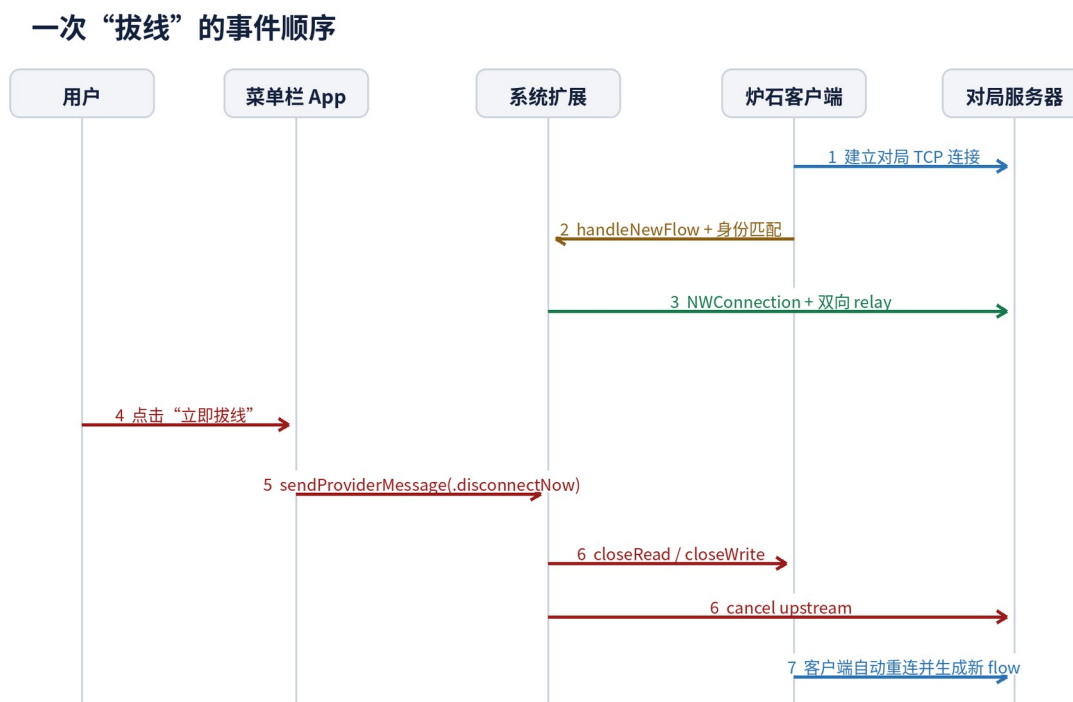


图 2 从目标 flow 建立到断开、再到客户端重连的顺序

## 5. 核心技术设计

### 5.1 Provider 启动与 flow-copying

startProxy 创建 NETransparentProxyNetworkSettings，首版只包含 outbound TCP 规则，并通过 setTunnelNetworkSettings 提交。规则需要覆盖 IPv4 与 IPv6。系统只会把规则启用后新建的 flow 交给 provider，因此首次启用后必须提示用户重启炉石或主动让游戏重新连接。<sup>[3]</sup>

当新 flow 到达时，provider 只对目标身份返回 true 并启动 relay；对非目标 flow 返回 false，让操作系统继续处理。这一点是 NETransparentProxyProvider 与普通 NEAppProxyProvider 的关键差异，也是 TavernBlink 能够在广义规则下保持应用隔离的基础。<sup>[4]</sup>

```
override fun handleNewFlow(_ flow: NEAppProxyFlow) -> Bool {
    guard let tcpFlow = flow as? NEAppProxyTCPFlow else { return false }
    guard matcher.matches(tcpFlow.metaData) else { return false }

    let relay = TCPFlowRelay(flow: tcpFlow)
    registry.insert(relay)
    relay.start()
    return true
}
```

**实现要求** 上面的代码只表示责任结构，不是可直接复制的完整实现。真实版本还必须处理 provider 尚未就绪、重复回调、relay 启动失败、registry 清理、扩展停止与 SDK API 可用性。

### 5.2 目标 App 身份识别

1. 主程序让用户选择 Hearthstone.app，或从已知安装位置发现候选；不直接假设路径。
2. 用 Security.framework 验证应用签名并读取 signing identifier；把 identifier 与展示名称保存到 App Group。
3. provider 从 flow.metaData.sourceAppSigningIdentifier 读取实际发起身份，并进行精确字符串比较。Apple 将该字段定义为来源应用的签名 identifier。
4. 若字段为空，记录 audit token 可用性并进入诊断，不自动扩大匹配范围。只有通过独立签名校验的 audit-token 回退完成后，才允许接管。
5. 游戏更新后重新验证目标应用签名；identifier 不变则继续，发生变化则要求用户确认。

不要把 bundle identifier 与 signing identifier 当作永远相同。对标准 Xcode 签名应用它们通常一致，但 TavernBlink 应从已安装二进制和实际 flow 两端各读取一次，并在诊断界面展示“预期值 / 实际值”。<sup>(g)</sup>

### 5.3 TCP relay 状态机

每条目标 flow 由独立 TCPFlowRelay 管理。建议使用一个专用串行队列或 Swift actor 保护状态，读写回调仍在 Network.framework 队列上执行。状态必须单向推进，任何终止路径都汇聚到一次幂等 close(reason:)。

```
created
-> connectingUpstream
-> openingClientFlow
-> relaying
-> closing
-> closed
```

- 上游：从 flow 的现代 remote endpoint 创建 NWConnection(to:using:)，参数使用 TCP。
- 客户端到服务器：readData → NWConnection.send；写完成后才继续下一次读，形成背压。
- 服务器到客户端：NWConnection.receive → flow.write；写完成后再 receive。
- 缓冲：每方向只允许一个在途块，建议块大小 64 KiB 起步；禁止无限累积 Data。
- 半关闭：任一方向 EOF 时停止该方向；错误或用户断开时关闭两端。
- 清理：所有 completion、state update、registry removal 只能执行一次。

NEAppProxyTCPFlow 提供 readData / write，NWConnection 提供到原始 endpoint 的 TCP 连接能力。由于 relay 只复制加密字节，TavernBlink 不需要证书、MITM 或应用层协议解析。<sup>(g)</sup>

### 5.4 “立即拔线”的关闭语义

主程序不直接访问 flow。它把版本化的 ProviderCommand.disconnectNow(requestID:) 编码为 Data，通过 NETunnelProviderSession.sendProviderMessage 发送；provider 在 handleAppMessage 中解析命令、原子快照当前目标 relay，并对每条 relay 执行幂等关闭。<sup>(g)</sup>

```
struct ProviderCommand: Codable {
    let protocolVersion: Int
    let requestID: UUID
    let action: Action
}

enum Action: String, Codable {
    case status
    case disconnectNow
}
```

1. 验证协议版本和 requestID；已处理的 requestID 直接返回原结果，防止重复点击。
2. registry 创建活动目标 relay 的不可变快照。
3. 每条 relay 调用 flow.closeReadWithError 与 flow.closeWriteWithError，再取消对应 NWConnection。
4. 从 registry 移除已关闭 relay，并返回 closedFlowCount、耗时和错误摘要。
5. 若关闭数为 0，UI 显示“未找到活动炉石连接”，而不是伪造成功。

Apple 文档要求 flow 使用完毕时关闭读写方向。这里利用同一生命周期操作主动结束代理 flow。需要实测的是炉石对 EOF/RST/连接取消的具体响应速度，而不是这些 API 是否存在。<sup>(7)</sup>

## 5.5 MVP 的“关全部”与后续的“选一条”

v0.1.0 应关闭所有由炉石签名身份发起且仍处于 relaying 状态的 TCP flow。这样做的优点是不会因为未知端口或多连接协议漏掉真正对局 flow；代价是可能同时关闭登录、遥测或内容服务连接。只要游戏能完整恢复，这个权衡适合验证核心价值。

积累本地诊断后，可为每条 flow 计算只用于排序的分数：连接年龄、最近活动、累计字节、远端口、断开后是否出现替代 flow。任何启发式都应可回退到“关全部”，并且不得上传远端地址或 payload。

5.6 故障处理与 fail-safe

表 5 关键故障与系统行为

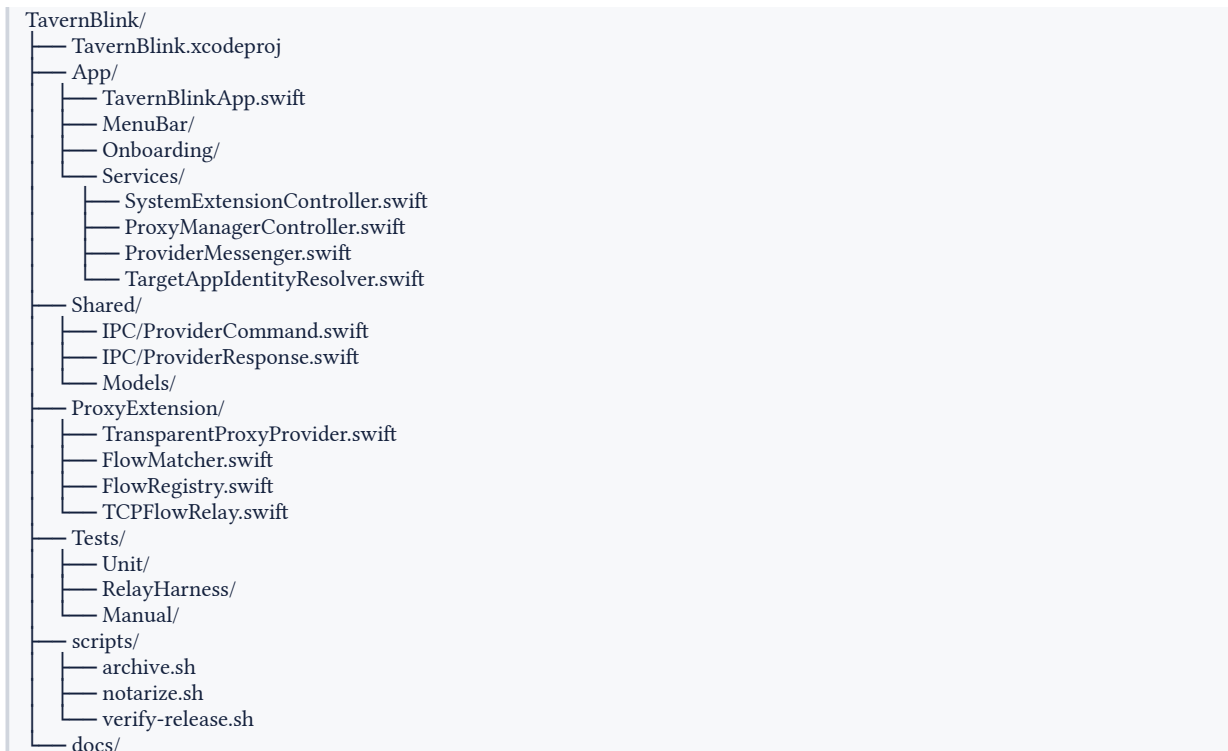
| 故障           | 期望行为                           | 用户提示       |
|--------------|--------------------------------|------------|
| 目标身份不匹配      | 返回 false，不接管 flow              | 未检测到炉石连接   |
| 上游连接失败       | 关闭该目标 flow，记录可读错误              | 连接建立失败，可重试 |
| provider 未运行 | 主程序刷新 manager 状态并给出启动入口        | 网络组件未启动    |
| 命令重复         | 按 requestID 幂等返回               | 沿用第一次结果    |
| relay 超时     | 终止单条 relay，不影响其他应用             | 目标连接超时     |
| 扩展升级待批准      | 保留旧扩展，等待系统审批后切换                | 需要在系统设置中批准 |
| 用户禁用网络扩展     | 停止显示 Ready，不自动反复开启             | 网络扩展已被用户关闭 |
| 主程序退出        | 可选让 provider 继续 relay；再次打开恢复状态 | 菜单栏重新连接中   |

5.7 Provider 生命周期与恢复规则

- startProxy 先提交网络设置，再加载目标身份；registry 每次启动都从空状态开始，不恢复旧 flow。
- stopProxy、系统扩展升级和用户禁用都进入同一关闭路径：停止接收新目标 flow、关闭全部 relay、只调用一次完成回调。
- 睡眠、唤醒、Wi-Fi 切换和上游路径变化依赖 NWConnection 状态推进；不使用无限定时重试，也不保留无界缓存。
- 非目标 flow 始终 fail-open 交回系统；只有 provider 已明确接管的目标 flow 才由 relay 生命周期负责关闭。
- 主程序必须提供 kill switch：停止 manager、禁用配置并刷新真实系统状态；失败时保留可复制的错误码和恢复步骤。

## 6. Xcode 工程与权限配置

### 6.1 推荐仓库结构



首版保持一个 Xcode project、两个可执行 targets（Host App 与 Network System Extension）和一个共享 Swift group。不要在 v0.1.0 引入独立 daemon、LaunchAgent 或第三方代理库。

### 6.2 Bundle IDs 与配置

- Host App：com.<team>.TavernBlink。
- System Extension：com.<team>.TavernBlink.ProxyExtension。
- App Group：group.com.<team>.TavernBlink。
- Bundle IDs 必须在 Apple Developer 账户中显式注册；<team> 只是占位符，不能直接提交。
- 主程序的 NETunnelProviderProtocol.providerBundleIdentifier 必须与系统扩展 bundle ID 完全一致。

6.3 Entitlements 最小集合

表 6 Host App 与 System Extension 的权限职责

| 目标               | Entitlement / Capability                            | 用途与约束                                        |
|------------------|-----------------------------------------------------|----------------------------------------------|
| Host App         | com.apple.developer.system-extension.install = true | 提交系统扩展激活、替换与停用请求                             |
| Host App         | Network Extensions capability（与 profile 一致）         | 创建并管理 transparent proxy 配置                   |
| Host App         | App Groups                                          | 与 provider 共享小型配置和状态                         |
| System Extension | app-proxy-provider-systemextension                  | Developer ID 分发下的 TCP/UDP app proxy provider |
| System Extension | App Sandbox                                         | 系统扩展运行要求与最小权限                                |
| System Extension | Outgoing Network Connections                        | 创建到原始目标的 NWConnection                        |
| System Extension | App Groups                                          | 读取目标 signing identifier 和写入诊断摘要              |

Apple 的 Network Extensions entitlement 文档列出 app-proxy-provider-systemextension，用于以 Developer ID 签名的系统扩展代理 TCP/UDP 流量；macOS capability 表也列出 Network Extensions 与 System Extension。实际 archive 中的 entitlements 必须与 provisioning profile 相交后仍保留这些值。

(10)

**最早验证** 在写 relay 前先 archive 一个最小 provider，并对 Host App 和 .systemextension 分别执行 codesign -d --entitlements :-。若 Release 构建缺少 entitlement，就立即修 App IDs、profiles 和 signing settings；不要等到发布周。

6.4 System Extension Info.plist

System Extension target 使用 NetworkExtension 字典和 NEProviderClasses 映射，把 com.apple.networkextension.app-proxy 指向 TransparentProxyProvider 主类。具体键名和生成后的 plist 必须以当前 Xcode SDK 模板为准，并在 archive 中检查，而不是只看 Xcode UI。

```
<key>NetworkExtension</key>
<dict>
  <key>NEProviderClasses</key>
  <dict>
    <key>com.apple.networkextension.app-proxy</key>
    <string>$(PRODUCT_MODULE_NAME).TransparentProxyProvider</string>
  </dict>
</dict>
```

6.5 Build configurations

- Debug：Development signing，详细 OSLog，允许诊断 UI；禁止把调试签名产物给测试者。
- Release：Developer ID Application，Hardened Runtime，独立 Developer ID provisioning profiles。
- Archive 必须包含 Contents/Library/SystemExtensions/<bundle>.systemextension。
- 支持 Apple Silicon 与 Intel：主程序和系统扩展都构建 arm64 + x86\_64。
- 建议最低 macOS 13，减少 UI 与 Swift concurrency 兼容分支；底层 API 虽从 macOS 11 可用，但不必首版承担全部旧系统测试。

7. 从零实施路线图

表 7 分阶段计划、产物与退出门槛

| 阶段                   | 预计    | 核心产物                                      | 退出门槛                       |
|----------------------|-------|-------------------------------------------|----------------------------|
| Phase 0<br>平台 PoC    | 3-5 天 | 可安装 provider、flow 日志、命令往返、真实断开测试          | 四个前置问题全部通过                 |
| Phase 1<br>工程骨架      | 2-3 天 | 菜单栏 App、controllers、共享 IPC、状态模型           | 安装/启动可重复，状态可恢复             |
| Phase 2<br>身份与规则     | 2-4 天 | 签名身份解析、matcher、TCP rules、非目标 pass-through | 只接管炉石 flow                 |
| Phase 3<br>TCP relay | 5-8 天 | 双向复制、背压、FlowRegistry、disconnectNow        | echo harness 与真实游戏稳定       |
| Phase 4<br>产品体验      | 3-5 天 | onboarding、错误引导、诊断导出、卸载说明                 | 第二位用户可独立安装                 |
| Phase 5<br>硬化测试      | 5-8 天 | 压力、睡眠/唤醒、IPv6、网络切换、性能与回归                  | 发布验收矩阵通过                   |
| Phase 6<br>发布        | 2-4 天 | CI、签名、公证、DMG、release notes                | 干净 Mac 安装和 Gatekeeper 检查通过 |



## 7.1 Phase 0：先证明路线

1. 注册 Host App、System Extension 和 App Group identifiers；创建开发与 Developer ID profiles。
2. 新建 macOS App target 与 Network System Extension target，添加最小 NETransparentProxyProvider。
3. 主程序提交 `OSSystemExtensionRequest.activationRequest`，完整处理 `.completed`、`.willCompleteAfterReboot`、`.needsUserApproval`、`.failed`。
4. 通过 `NETransparentProxyManager.loadAllFromPreferences` 复用或创建唯一配置，设置 protocol、bundle ID、描述和 enabled，再保存并启动。
5. provider 只设置 TCP included rules，所有 `handleNewFlow` 暂时返回 false，确认普通网络不受影响。
6. 增加仅元数据日志，运行炉石并记录 actual signing identifier、协议、远端端点类型和 flow 生命周期。
7. 只对已确认 identifier 返回 true，先用本地 echo server 验证 relay，再进入真实对局。
8. 实现最小 `disconnectNow`，执行 20 次真实对局断开；记录关闭数量、重连时间和失败模式。

**Phase 0 交付物** 一个不追求漂亮 UI 的 development build、一份 20 次测试结果表、archive entitlements 输出和一页 Go/No-Go 决策。只有这四样同时存在，才算路线被证明。

## 7.2 Phase 1-3：把 PoC 变成可维护内核

- 把安装、manager 配置、provider messaging 分成独立 service；所有异步 API 包装为可测试状态转换。
- 建立版本化 IPC schema；未知 command 或版本必须返回显式错误。
- FlowMatcher 纯函数化；用固定 metadata fixtures 做单元测试。
- TCPFlowRelay 不依赖 UI；用协议抽象 flow/connection 后建立 fake 与真实 echo harness。
- FlowRegistry 用 actor 或串行队列保证 insert/remove/disconnectAll 原子性。
- 所有关闭路径统一到 `close(reason:)`，消灭 completion double-call 和 relay 泄漏。

7.3 Phase 4-6：从能跑到能交付

- 首次启动用清晰步骤解释为什么需要两次系统许可，以及下一步去哪里批准。
- 建立状态面板和诊断导出；默认日志只包含事件、错误码和匿名 flow 序号。
- 在至少两台物理 Mac 上完成系统版本与芯片矩阵，不把 CI runner 当作系统扩展实测。
- 把签名、公证、staple、验证和 DMG 生成脚本化；tag 才触发 release archive。
- Beta 先小规模封闭测试，收集重连成功率；未确认 Blizzard 风险前不公开宣传“安全”。

7.4 第一周的每日任务

表 8 第一周建议节奏

| 日     | 任务                                                     | 当天验收                           |
|-------|--------------------------------------------------------|--------------------------------|
| Day 1 | 创建 identifiers、profiles、Xcode 两 target、entitlements    | Debug/Release 签名设置有记录          |
| Day 2 | 激活 system extension；处理用户批准                             | systemextensionsctl list 可见并启用 |
| Day 3 | 保存/启动 NETransparentProxyManager；provider start/stop 日志 | 主程序显示真实 NE 状态                  |
| Day 4 | 全 TCP rule + flow-copying；记录炉石 signing identifier      | 浏览器正常，炉石 flow 可区分              |
| Day 5 | provider messaging round-trip；最小 relay/close           | 按钮到 provider 有带 requestID 回执   |
| 周末    | 本地 echo + 真实对局 20 次测试                                  | 形成 Go/No-Go 结论                 |

## 8. 测试与验收

### 8.1 测试金字塔

- 纯单元测试：IPC Codable、版本兼容、FlowMatcher、FlowRegistry 幂等、状态 reducer。
- relay harness：本地 TCP echo/slow server 模拟部分写、EOF、延迟、错误与大流量。
- provider 集成测试：真实 Mac 上安装、启动、观察 flow、发送命令；这部分不能由 simulator 代替。
- 游戏场景测试：大厅、进入战斗、动画中、招募阶段、掉线中重复点击、网络本身不可用。
- 发布验收：干净账户、第二台 Mac、离线 Gatekeeper、卸载与重装。

### 8.2 Relay 必测用例

表 9 TCP relay 关键用例

| 用例            | 故障注入                   | 通过标准                     |
|---------------|------------------------|--------------------------|
| 小包双向          | 1-256 B 请求/响应          | 字节完全一致，无额外等待             |
| 大流量           | 连续 500 MB              | 内存保持有界，无数据损坏             |
| 慢上游           | 每块延迟 100-500 ms        | 不无限排队，UI 仍可响应            |
| 部分写           | 分段 completion          | 不丢字节、不重发                 |
| 客户端 EOF       | 先关闭 client write       | 上游收到 EOF，relay 正常收尾      |
| 服务器 EOF       | 上游先结束                  | client flow 正常关闭         |
| 用户断开          | relay 中途 disconnectNow | 两端在目标时限内关闭               |
| 重复断开          | 相同 requestID 发送 10 次   | 只执行一次，回执一致               |
| 并发 flow       | 20 条目标连接               | registry 数量准确，全部可清理      |
| provider stop | relay 中停止扩展            | 所有 completion 收敛，无长期网络影响 |

## 8.3 真实环境矩阵

- 芯片：至少一台 Apple Silicon；发布 Universal 2 前再验证一台 Intel。
- 系统：最低支持版本、当前稳定版本、下一主版本；beta 仅用于兼容预警，不作为首发支持承诺。
- 网络：Wi-Fi、手机热点、IPv4、可用时的 IPv6、睡眠/唤醒、Wi-Fi 切换。
- 并行应用：Safari/Chrome 下载、Discord/Zoom 通话、Git clone、流媒体。
- 炉石状态：刚启动、登录、排队、战斗、招募、结算、主动退出。

## 8.4 游戏特定验收记录

每次测试至少记录：测试编号、macOS build、炉石版本、目标 flow 数、点击时间、provider 收到时间、关闭完成时间、新 flow 出现时间、游戏恢复时间、是否跳过动画、是否出现登录失败。任何失败必须保存 OSLog 摘要，但不保存 payload。

**发布阈值** 建议 Beta 前完成至少 100 次断开，其中有效战斗场景成功率  $\geq 95\%$ ，且非炉石网络回归 0 个可复现中断。若成功率不足，先分析 flow 选择与关闭顺序，不要用更频繁重试掩盖问题。

## 8.5 可观测性

- 统一 Logger categories：activation、manager、messaging、matching、relay、disconnect、release。
- 每个 relay 使用随机本地 flow ID；不把远端 IP 当作长期 identifier。
- 指标只在本机汇总：activeFlowCount、bytesRelayed、relayAge、lastDisconnectDuration、errorCode。
- 默认不上传；导出诊断包前向用户展示包含内容并允许取消。
- payload、Battle.net token、账号名、完整路径和私有地址不进入日志。

# 9. 首次安装与产品体验

## 9.1 首次启动流程

1. 欢迎页说明功能、需要系统网络扩展以及 Blizzard 风险，不在用户不知情时启用。

2. 用户点击“安装网络组件”，主程序提交系统扩展激活请求。
3. 若系统要求批准，界面显示当前请求状态和系统提供的设置入口；批准后自动刷新。
4. 主程序创建/复用 TavernBlink 的 transparent proxy 配置，保存并启用；系统可能再次询问网络配置权限。
5. 定位并验证 Hearthstone.app，显示 signing identifier 的验证结果。
6. 启动 proxy，等待 manager 进入 connected；提示重启炉石以确保产生新 flow。
7. 检测到目标 flow 后，菜单栏变为 Ready，按钮才启用。

System Extension 的首次激活可能需要用户在系统设置中批准。Apple 的 SystemExtensions API 负责激活、更新和停用扩展；应用必须正确处理“等待用户批准”与“需要重启后完成”，而不是用固定延时猜测。<sup>(41)</sup>

9.2 UI 状态机

表 10 菜单栏状态与允许操作

| 状态                   | 界面文案                  | 允许操作       |
|----------------------|-----------------------|------------|
| needsInstall         | 尚未安装网络组件              | 安装         |
| needsApproval        | 请在系统设置中批准 TavernBlink | 打开设置、刷新    |
| configurationMissing | 尚未创建网络代理配置            | 完成设置       |
| starting             | 网络组件启动中               | 取消等待、查看诊断  |
| readyNoFlow          | 已就绪，等待炉石连接            | 打开炉石、刷新    |
| readyWithFlow        | 检测到炉石连接               | 立即拔线       |
| disconnecting        | 正在关闭连接                | 禁用重复按钮     |
| success              | 已关闭 N 条连接，等待游戏重连      | 查看状态       |
| error                | 操作失败：可读原因             | 重试、导出诊断、禁用 |

9.3 防误触设计

- 只有 manager connected 且 activeFlowCount > 0 时启用主按钮。

- 一次命令进行中禁用按钮；完成后设置 1-2 秒冷却。
- 默认不注册全局快捷键；后续增加时必须允许关闭和修改。
- 菜单中始终提供“禁用网络组件”和“导出诊断”。
- 断开失败不自动关闭网卡、不循环重试、不扩大到其他进程。

## 10. CI/CD 与发布

### 10.1 CI 能验证什么

- 每个 PR：SwiftFormat/SwiftLint（若采用）、纯单元测试、relay harness、Debug build。
- main：Release build、静态检查、archive 结构检查、entitlements diff。
- tag：导入临时签名 keychain、archive/export、创建 DMG、公证、staple、生成 checksums。
- CI 不能替代：系统扩展用户批准、真实 flow 拦截、真实炉石重连、睡眠/唤醒。

### 10.2 签名与公证

直接分发的 Mac App 使用 Developer ID Application certificate；Apple 的说明指出，Developer ID 签名配合 notarization ticket，使 Gatekeeper 能验证软件未被已知恶意软件污染且未被篡改。系统扩展属于高级 capability，还需要匹配的 Developer ID provisioning profiles。<sup>[12]</sup>

Apple 提供 notarytool 上传软件，并用 stapler 把票据附加到分发产物。公证是自动恶意内容与签名检查，不是 App Review，也不代表对 TavernBlink 用途的批准。<sup>[13]</sup>

```
xcodebuild archive -scheme TavernBlink -configuration Release -archivePath build/TavernBlink.xcarchive

xcodebuild -exportArchive -archivePath build/TavernBlink.xcarchive -exportOptionsPlist ExportOptions.plist -exportPath build/export

xcrun notarytool submit build/TavernBlink.dmg --keychain-profile "TavernBlink-Notary" --wait

xcrun stapler staple build/TavernBlink.dmg
xcrun stapler validate build/TavernBlink.dmg
codesign --verify --deep --strict --verbose=2 build/export/TavernBlink.app
spctl --assess --type execute --verbose=4 build/export/TavernBlink.app
```

### 10.3 Release pipeline

1. 创建 vX.Y.Z tag；CI 只从已保护的 tag 构建。

- 2. 导入 Developer ID certificate、profiles 和 App Store Connect API key；全部存为加密 secrets。
- 3. Archive 后先检查 .systemextension 是否存在，再检查 host/provider 的实际 entitlements。
- 4. 创建并签名 DMG，提交 notarization，等待 Accepted，再 staple。
- 5. 运行 codesign、stapler、spctl 验证，计算 SHA-256。
- 6. 在物理测试 Mac 上下载 CI 产物，完成首次安装、批准、真实断开、禁用、卸载。
- 7. 发布 GitHub Release：DMG、checksum、release notes、已知问题、风险说明和卸载步骤。

10.4 更新与卸载

- v0.x 手动下载更新；稳定后再评估 Sparkle 2，避免首版扩大签名面。
- 系统扩展更新使用 activation request 的 replacement 流程，正确返回 replace/cancel。
- 卸载前先 stop manager、禁用并移除配置，再提交 system extension deactivation。
- Finder 直接删除 App 不应是唯一卸载路径；应用内提供明确“完全卸载”流程。
- 配置删除与 extension deactivation 都是有影响操作，必须由用户明确触发并展示结果。

10.5 发布物清单

表 10-1 每个公开测试版本应附带的交付物

| 产物                    | 内容                                        | 发布前验证                      |
|-----------------------|-------------------------------------------|----------------------------|
| TavernBlink.dmg       | 签名、公证并 staple 的 Universal 2 安装包           | spctl、stapler、codesign 均通过 |
| SHA256SUMS            | DMG 的 SHA-256 摘要                          | 在 CI 与下载后的文件上结果一致          |
| Release notes         | 变更、兼容系统、已知问题、账号风险                         | 版本号与 tag、DMG 一致            |
| README / 隐私 / 卸载      | 授权说明、本地数据边界、禁用与完全卸载                       | 第二位用户可离线完成操作               |
| Entitlements snapshot | Host App 与 .systemextension 的 codesign 输出 | 与批准的 profiles 和基线 diff 一致  |
| Notarization evidence | Submission ID、Accepted 状态及日志归档            | 可追溯到本次 release archive     |

# 11. 风险与合规

## 11.1 技术风险登记

表 11 主要风险、等级与缓解措施

| 风险                     | 等级 | 早期信号                        | 缓解 / 决策                     |
|------------------------|----|-----------------------------|-----------------------------|
| Release entitlement 缺失 | 高  | archive 中 capability 被剥离    | Phase 0 最先 archive；不通过则暂停   |
| 关闭不触发快速重连              | 高  | 游戏长时间卡住或退出登录                | 20 次实测 Go/No-Go；不通过则停止路线    |
| 广义 TCP rules 影响其他 App  | 高  | Safari/系统服务异常               | 严格 return false；全套网络回归；一键禁用 |
| relay 内存或背压错误          | 高  | 内存持续增长、吞吐下降                 | 每方向单块在途；大流量与慢端压力测试          |
| 目标身份不可用/变化             | 中  | signing identifier 为空或更新后变化 | 静态签名验证 + 显式重新确认             |
| 炉石改用 UDP/多连接           | 中  | 断开 TCP 无效或成功率下降             | 诊断协议分布；仅证实后实现 UDP           |
| macOS 更新改变许可流程         | 中  | beta 上安装/状态异常               | 支持矩阵、预发布测试、清晰错误引导           |
| 用户误以为工具官方安全            | 高  | 公开宣传出现 “零风险”                | 明确非官方、账号风险，不做保证             |
| 签名凭据泄露                 | 高  | CI secrets 或证书外泄            | 临时 keychain、最小权限、轮换与审计      |

## 11.2 Blizzard 规则与账号风险

Blizzard EULA 对未经授权的软件、作弊以及拦截/收集平台信息存在广泛限制。TavernBlink 虽不解析 payload、不注入游戏，但其目的仍是通过非正常断线改变可用操作时间，不能据此推导“不会违规”或“不会封号”。<sup>(14)</sup>

- README、首次启动和 release notes 都应写明：非 Blizzard 官方工具，使用者自担账号风险。
- 不要声称“安全”“不会检测”“不会封号”，也不要设计绕过检测的能力。
- 不要收集用户 Battle.net 凭据、账号标识或游戏 payload。



- 公开大规模分发、收费或营销前应另做法律与平台政策评估。
- 如果 Blizzard 明确禁止或要求停止，应准备下架发布物并停止维护相关功能。

11.3 Apple 发布边界

Apple notarization 只验证恶意内容与签名完整性，不等于 App Store 审核，也不替 TavernBlink 解决第三方游戏条款。首发选择 Developer ID 直接分发是工程上更现实的渠道，但仍必须遵守 entitlement、Hardened Runtime、系统扩展许可与用户透明度要求。

**发布建议** 技术落地与公开发行业应分成两个决策。先完成自用 Alpha 和封闭 Beta，证明稳定性；再基于 Blizzard 风险、用户沟通与法律意见决定是否公开发布。报告不把“能 notarize”当成“可以无风险推广”。

12. 工期、资源与 Definition of Done

12.1 现实工期

对熟悉 Swift、Network.framework 和 macOS signing 的单人开发者，完整 Beta 约 4-6 周；对第一次开发 Network Extension 的开发者，更现实的是 6-9 周、约 100-160 小时。Phase 0 的结果会显著改变估算，因此不要在 PoC 前承诺公开发布日期。

表 12 六周主计划与两周缓冲

| 周          | 重点                                  | 里程碑            |
|------------|-------------------------------------|----------------|
| Week 1     | entitlement、安装、manager、flow 身份、最小断开 | Go/No-Go       |
| Week 2     | 工程重构、IPC、matcher、echo harness       | 可重复开发构建        |
| Week 3     | 完整 TCP relay、registry、幂等关闭          | Alpha 内核       |
| Week 4     | onboarding、状态机、诊断、错误恢复              | 可交给第二位用户       |
| Week 5     | 压力、网络切换、睡眠/唤醒、游戏 100 次测试            | Beta candidate |
| Week 6     | CI、Developer ID、notarization、DMG、文档 | v0.2.0 Beta    |
| Buffer 1-2 | macOS/游戏差异、签名问题、UDP 调研（仅如需要）        | 延迟风险吸收         |

## 12.2 所需资源

- 一台主要开发 Mac；至少再有一个干净用户账户，最好有第二台物理 Mac。
- Apple Developer Program 付费会员与 Developer ID Application certificate。
- GitHub repository 与 macOS CI minutes；首版不需要服务器或数据库。
- 可重复的 Hearthstone 测试账号与人工测试记录；不要把主账号当作唯一测试资产。
- 时间预算优先给 signing、relay 和真实测试，UI polish 排在其后。

## 12.3 v0.2.0 Beta 的 Definition of Done

- Release archive 同时包含正确签名的 Host App 与 .systemextension。
- 实际 entitlements 与 provisioning profiles 已保存为发布证据。
- 首次安装、两次系统许可、启动、检测炉石 flow、断开、禁用、卸载都有明确 UX。
- 100 次游戏断开达到约定成功率；所有失败都有可解释日志。
- 非炉石应用回归无可复现网络中断。
- 2 小时稳定性、500 MB relay、20 flow 并发和 100 次重复命令测试通过。
- DMG 通过 notarytool、stapler、codesign 和 spctl 验证。
- README、隐私说明、已知问题、EULA 风险和卸载步骤完成。
- 从 GitHub Release 下载到第二台 Mac 的全流程通过。

## 12.4 建议立即创建的十个 GitHub Issues

1. Bootstrap host app and network system extension targets
2. Validate Developer ID entitlements in archived products
3. Implement system extension activation state machine
4. Create and start NETransparentProxyManager configuration
5. Log flow metadata and confirm Hearthstone signing identifier
6. Implement versioned provider messaging
7. Build bounded TCP flow relay with echo harness
8. Add FlowRegistry and idempotent disconnectNow
9. Run 20-match Go/No-Go experiment
10. Automate signed, notarized DMG release verification

**下一步** 先做 Week 1 / Phase 0，不要先画完整设置页。最值得创建的第一个 branch 是 spike/transparent-proxy-entitlements；它的唯一目标是把一个最小 provider 以正确 entitlement 启动起来，并生成可核验的 archive。

附录 A：关键 API 映射

表 A-1 TavernBlink 需求与 Apple API

| 需求          | 主要 API                                               | 验证重点                               |
|-------------|------------------------------------------------------|------------------------------------|
| 安装系统扩展      | OSSystemExtensionRequest                             | 审批、替换、重启后完成、失败回调                   |
| 配置透明代理      | NETransparentProxyManager                            | load/save/enable/start 与唯一配置       |
| 设置拦截规则      | NETransparentProxyNetworkSettings /<br>NENetworkRule | outbound TCP、IPv4/IPv6             |
| 接收新 flow    | handleNewFlow                                        | 目标 true，非目标 false                  |
| 识别来源        | NEFlowMetaData.sourceAppSigningIdentifier            | 预期值与实际值一致                          |
| 代理客户端       | NEAppProxyTCPFlow                                    | open、readData、write、close          |
| 连接远端        | NWConnection                                         | endpoint、state、send/receive、cancel |
| 主程序命令       | NETunnelProviderSession.sendProviderMessage          | 版本、requestID、provider 未运行          |
| provider 处理 | handleAppMessage                                     | 解析、幂等、回执                           |
| 签名分发        | Developer ID + notarization                          | profiles、Hardened Runtime、staple   |

附录 B：参考资料

以下资料于 2026-07-28 核验。Apple Developer Documentation 页面可能随 SDK 更新；实现时应同时以当前 Xcode SDK 的 Swift interface 和编译器提示为准。

- [1] [z2z63/hearthstone\\_skipper README](#)。说明 Clash/Mihomo、TUN、进程识别要求及断线跳动画原理。
- [2] [hearthstone\\_skipper/src/skipper.cpp](#)。展示 GET connections、按 processPath/host 筛选并发送 DELETE 的实现。
- [3] [Apple — NETransparentProxyProvider](#)。透明代理 provider 类与 macOS 可用性。

- 〔4〕 [Apple — Handling Flow Copying](#)。说明返回 false 可让操作系统处理 flow，并给出 flow-copying 模式。
- 〔5〕 [Apple — NETransparentProxyManager](#)。配置与控制 transparent proxy。
- 〔6〕 [Apple — sourceAppSigningIdentifier](#)。来源应用签名 identifier 元数据。
- 〔7〕 [Apple — NEAppProxyFlow](#)。flow 生命周期及关闭读写方向的方法。
- 〔8〕 [Apple — NWConnection](#)。到 NWEndpoint 的连接、发送、接收和状态管理。
- 〔9〕 [Apple — sendProviderMessage / handleAppMessage](#)。主程序与 provider 的系统通信通道。
- 〔10〕 [Apple — Network Extensions Entitlement](#)。包含 app-proxy-provider-systemextension entitlement 值。
- 〔11〕 [Apple — Installing System Extensions and Drivers](#)。系统扩展激活、更新和停用流程。
- 〔12〕 [Apple — Developer ID Certificates](#)。Mac App 直接分发签名与 Developer ID 证书说明。
- 〔13〕 [Apple — Notarizing macOS software before distribution](#)。notarytool、stapler 与公证 workflow。
- 〔14〕 [Blizzard End User License Agreement](#)。未经授权软件、作弊及信息拦截相关规则；发布前需重新核验最新版。
- 〔15〕 [Apple — Supported capabilities \(macOS\)](#)。macOS profiles 支持的 Network Extensions 与 System Extension capabilities。

## 报告结论

TavernBlink 选择 NETransparentProxyProvider 是正确的长期路线：它能把“第三方代理核心持有并删除连接”变成“应用自己的系统扩展持有并关闭目标 flow”，从产品形态上消除 Clash 依赖。落地关键是严格控制顺序：先证明 entitlement 与真实重连，再完成 relay 内核，再做用户体验与发布。只要 Phase 0 四项门槛全部通过，这个项目可以按六周主计划推进到可签名、公证和小规模测试的 Beta。